

Learning Objectives

Part 1: AI/ML Basics for ITOps (30 min)

- 1.1 Why AI Entered IT Operations
- 1.2 Core Machine Learning Concepts (Refresher)
 - Supervised vs. Unsupervised Learning
 - Training vs. Inference
 - Model Drift (important operational concept)
- 1.3 Why Traditional Static Thresholds Fail
- 1.4 Class Discussion (5 min)

Part 2: Anomaly Detection (30 min)

- 2.1 What Is Anomaly Detection?
- 2.2 Types of Anomalies
- 2.3 Common Anomaly Detection Techniques (Conceptual Overview)
 - Statistical Methods
 - Machine Learning Methods
 - Dynamic Baseline (the practical AIOps term)
- 2.4 Worked Example: Dynamic Baseline vs. Static Threshold
- 2.5 Evaluating Anomaly Detection: Precision, Recall, and the Trade-off
- 2.6 Class Exercise (5 min)

Part 3: Time-Series Forecasting (25 min)

- 3.1 What Is Time-Series Forecasting?
- 3.2 Why Forecasting Matters — Moving from Reactive to Proactive
- 3.3 Core Time-Series Concepts
- 3.4 Common Forecasting Approaches (Conceptual Level)
- 3.5 Worked Example: Capacity Forecasting
- 3.6 Limitations of Forecasting
- 3.7 Class Exercise (5 min)

Part 4: Intelligent Alerting (25 min)

- 4.1 The Problem: Alert Fatigue
- 4.2 Key Techniques in Intelligent Alerting
 - Dynamic Thresholds (from anomaly detection)
 - Alert Suppression & Deduplication
 - Severity Scoring / Prioritization
 - Predictive Alerting
 - Alert Routing & Escalation Intelligence
- 4.3 Static Alerting vs. Intelligent Alerting — Side-by-Side
- 4.4 Risks and Limitations to Discuss
- 4.5 Class Exercise (5 min)

Part 5: Recap & Discussion (10 min)

- Key Takeaways
- Discussion Questions for Wrap-Up
- Bridge to Lecture 3

Learning Objectives

By the end of this session, students will be able to:

1. Explain what AIOps is and why traditional threshold-based monitoring breaks down at scale.

2. Understand foundational AI/ML concepts relevant to IT operations (supervised vs. unsupervised learning, training vs. inference).
3. Explain how anomaly detection algorithms identify abnormal system behavior without hardcoded thresholds.
4. Understand time-series forecasting and its use in capacity planning and proactive alerting.
5. Describe intelligent alerting and how it reduces noise compared to traditional static-threshold alerting.

Session Time Allocation (120 minutes)

Segment	Topic	Duration
1	AI/ML Basics for ITOps	30 min
2	Anomaly Detection	30 min
3	Time-Series Forecasting	25 min
4	Intelligent Alerting	25 min
5	Recap, Discussion & Q&A	10 min

Part 1: AI/ML Basics for ITOps (30 min)

1.1 Why AI Entered IT Operations

Recall from Lecture 1: modern infrastructure produces enormous volumes of metrics, logs, and traces. A single mid-sized enterprise environment can generate millions of data points and thousands of alerts per day. Humans cannot manually review this volume. This gave rise to **AIOps (Artificial Intelligence for IT Operations)** — the application of machine learning and big-data analytics to automate and improve IT operations tasks such as:

- Detecting abnormal behavior automatically (anomaly detection)
- Predicting future resource needs (forecasting)
- Reducing alert noise (intelligent alerting)
- Correlating related events into a single incident (Lecture 3)
- Suggesting or automating root cause diagnosis

Important framing for students: AIOps does not replace engineers — it changes their role from “constantly watching dashboards and reacting to every alert” to “reviewing prioritized, context-rich incidents and making decisions.”

1.2 Core Machine Learning Concepts (Refresher)

Supervised vs. Unsupervised Learning

Type	Definition	ITOps Example
Supervised Learning	Model learns from labeled historical data (input → known correct output)	Predicting if an incident will be a “P1” based on past incidents labeled by severity
Unsupervised Learning	Model finds patterns in unlabeled data — no “correct answer” given	Anomaly detection: learning what “normal” looks like without being told which historical points were anomalies

Most AIOps anomaly detection use cases are **unsupervised** or **semi-supervised**, because most organizations do not have a labeled dataset of “here are all our past anomalies” — anomalies are, by definition, rare and unlabeled.

Training vs. Inference

- **Training:** the model learns patterns from historical data (e.g., “here’s 3 months of CPU metrics”).
- **Inference:** the trained model is applied to new, live data to make a prediction or decision (e.g., “is this new CPU reading normal or abnormal?”).

Model Drift (important operational concept)

Because infrastructure and application behavior change over time (new deployments, seasonal traffic, changing user behavior), a model trained once will gradually become less accurate. This is called **model drift**, and it’s why production AIOps systems continuously retrain models on a rolling window of recent data rather than training once and never updating.

1.3 Why Traditional Static Thresholds Fail

Before AI, the dominant monitoring approach was: “Alert if metric X crosses threshold Y” (e.g., “alert if CPU > 80%”).

Problems with static thresholds:

1. **One size doesn’t fit all** — a database server might normally run at 85% CPU as its baseline, while a web server at 85% CPU is abnormal. A single global threshold can’t capture this.
2. **No awareness of time-based patterns** — traffic on an e-commerce site is naturally higher on weekday evenings; a static threshold either causes false alarms during normal peak hours or misses real problems during quiet hours.
3. **Alert fatigue** — engineers become desensitized to alerts when too many are false positives, leading to real issues being missed (“the boy who cried wolf” problem).
4. **Manual maintenance burden** — thresholds must be manually tuned for every single metric, on every single system, and re-tuned as systems change.

This is precisely the gap that **AI-based anomaly detection** fills — by learning what “normal” looks like *for that specific metric, on that specific system, at that specific time of day/week* — rather than relying on a single fixed number.

1.4 Class Discussion (5 min)

Ask: “Can anyone share an example — from an internship, project, or even a home device — where a fixed alert threshold caused annoying false alarms, or missed a real problem?” This grounds the abstract concept in relatable experience before moving into the technical detail.

Part 2: Anomaly Detection (30 min)

2.1 What Is Anomaly Detection?

Anomaly detection is the process of identifying data points, events, or patterns that deviate significantly from expected/normal behavior, without relying on a fixed, manually-set threshold.

2.2 Types of Anomalies

Type	Description	Example
Point anomaly	A single data point is abnormal compared to the rest	A sudden CPU spike to 100% for one minute
Contextual anomaly	A value is normal in one context but abnormal in another	500 requests/sec is fine on Black Friday, but abnormal on a random Tuesday at 3 AM
Collective anomaly	A group of data points together is	A slow, steady memory increase over

	abnormal, even if each individual point looks fine	6 hours (a memory leak) — no single point looks alarming
--	--	--

2.3 Common Anomaly Detection Techniques (Conceptual Overview)

Students do not need to implement these mathematically in this course, but should understand the intuition:

Statistical Methods

- **Standard deviation / Z-score based:** flag a point if it's more than N standard deviations from the historical mean.
- **Moving average / Exponential smoothing:** compare current value against a smoothed recent baseline.
- **Seasonal decomposition:** separate a metric into trend + seasonality + residual (noise); flag large residuals.

Machine Learning Methods

- **Clustering-based (e.g., k-means, DBSCAN):** normal data points cluster together; points far from any cluster are anomalies.
- **Isolation Forest:** builds random decision trees; anomalies are “isolated” in fewer splits than normal points because they're rare and different.
- **Autoencoders (Neural Networks):** a neural network learns to compress and reconstruct normal data; when it's fed an anomaly, the reconstruction error is high, flagging it.

Dynamic Baselining (the practical AIOps term)

Most commercial AIOps tools (e.g., Moogsoft, Dynatrace, Splunk ITSI, BigPanda) use a technique broadly called **dynamic thresholding** or **dynamic baselining**: the system continuously learns a metric's “normal band” (which can vary by hour of day and day of week) and only alerts when a value falls meaningfully outside that adaptive band — directly solving the static-threshold problems from Part 1.

2.4 Worked Example: Dynamic Baseline vs. Static Threshold

Imagine CPU usage on a batch-processing server that normally: - Runs at ~20% CPU most of the day - Spikes to ~75% CPU every night at 2 AM during scheduled batch jobs

Static threshold approach: Set alert at “CPU > 70%.” - Result: Every single night at 2 AM, a false alert fires — a completely normal, expected event.

Dynamic baseline approach: The model learns that 75% CPU at 2 AM is the *expected* pattern. - Result: No alert at 2 AM (expected behavior). If CPU instead spikes to 75% at 2 PM (an unexpected time) or fails to rise at all at 2 AM (a missed batch job — a *collective/contextual* anomaly), the system flags it appropriately.

2.5 Evaluating Anomaly Detection: Precision, Recall, and the Trade-off

This is a good moment to reinforce (or introduce) two key ML evaluation concepts in the ITOps context:

Term	Definition	ITOps Meaning
False Positive	Flagged as anomaly, but actually normal	Wastes engineer time investigating a non-issue
False Negative	Missed anomaly, actually abnormal	A real outage goes undetected — potentially worse
Precision	Of all flagged anomalies, how many were real	High precision = fewer false alarms
Recall	Of all real anomalies, how many were caught	High recall = fewer missed incidents

Key teaching point: There is almost always a trade-off between precision and recall — tuning a model to be more sensitive catches more real issues (higher recall) but also raises more false alarms (lower precision), and vice versa. Production AIOps tuning is fundamentally about finding the right balance for the organization’s risk tolerance.

2.6 Class Exercise (5 min)

Show a simple line chart (hand-drawn or verbally described) of CPU usage over a week with one obvious weekday pattern and one clear outlier. Ask students to identify: (a) is this a point, contextual, or collective anomaly, and (b) would a static threshold have caught it?

Part 3: Time-Series Forecasting (25 min)

3.1 What Is Time-Series Forecasting?

Time-series forecasting uses historical, time-ordered data to predict future values. In ITOps, this typically means predicting future resource usage, traffic, or capacity needs.

Examples of what gets forecasted: - Future disk usage (“At this growth rate, this volume will be full in 12 days”) - Future traffic/request volume (“Expect 3x normal traffic during the upcoming sale event”) - Future memory usage trend (to catch a slow leak before it causes an outage)

3.2 Why Forecasting Matters — Moving from Reactive to Proactive

This is the conceptual centerpiece of the AIOps value proposition:

Traditional (Reactive)	AIOps-Enabled (Proactive)
Disk fills up → application crashes → incident raised	Forecast shows disk will fill in 5 days → ticket auto-created to expand storage before impact
Traffic spike overwhelms servers → downtime	Forecast predicts spike based on historical pattern → auto-scaling triggered in advance

3.3 Core Time-Series Concepts

- **Trend** — the long-term direction of the data (e.g., steadily increasing memory usage).
- **Seasonality** — repeating patterns at fixed intervals (daily, weekly, monthly) — e.g., traffic peaks every weekday at noon.
- **Residual/Noise** — the random variation left after removing trend and seasonality.
- **Stationarity** — a time series whose statistical properties (mean, variance) don’t change over time; many forecasting models assume or require this, or transform the data to achieve it.

3.4 Common Forecasting Approaches (Conceptual Level)

Method	Intuition	Good For
Moving Average	Simple average of recent past values	Short-term, low-complexity trends
ARIMA (AutoRegressive Integrated Moving Average)	Statistical model combining past values + past errors	Metrics with clear trend/seasonality, moderate historical data
Exponential Smoothing (Holt-Winters)	Weighted average favoring recent data, explicitly models trend + seasonality	Metrics with strong daily/weekly seasonality (e.g., web traffic)
Prophet (Facebook/Meta)	Decomposes into trend + seasonality + holiday effects; robust to missing data	Business-style time series with known events (holidays, sales)

Students don't need to implement these; the goal is recognizing the names and knowing when each is appropriate.

3.5 Worked Example: Capacity Forecasting

Scenario: A database volume has grown from 400GB to 720GB over the last 90 days, fairly linearly, with no signs of stopping. Total volume capacity is 1TB.

Forecast calculation (simple linear trend, illustrative): - Growth rate: $(720 - 400) \text{ GB} / 90 \text{ days} \approx 3.6 \text{ GB/day}$ - Remaining capacity: $1000 - 720 = 280 \text{ GB}$ - Days until full: $280 / 3.6 \approx \mathbf{78 \text{ days}}$

Operational action: An AIOps system would automatically generate a proactive ticket: "Volume X projected to reach capacity in ~78 days at current growth rate — recommend expansion or archival policy review." This is a direct connection back to the "Volume Expansion" Day-2 storage operations concept from real production environments (as seen in infrastructure/CSI contexts).

3.6 Limitations of Forecasting

- Forecasts assume the future resembles the past — sudden business changes (a marketing campaign, a new feature launch) can invalidate a forecast.
- Forecasting accuracy degrades further into the future ("forecast horizon") — a 7-day forecast is far more reliable than a 6-month forecast.
- Requires sufficient historical data — a new service with only 3 days of data cannot be reliably forecasted yet.

3.7 Class Exercise (5 min)

Give students a simple table of weekly disk usage numbers over 4 weeks. Have them calculate a simple linear growth rate and estimate days-to-full, then discuss what business changes might make that forecast wrong.

Part 4: Intelligent Alerting (25 min)

4.1 The Problem: Alert Fatigue

Modern environments can generate **thousands of alerts per day**. Studies and industry reports commonly cite that the majority of alerts in large enterprises are either duplicates, low-priority, or false positives. This leads to:

- Engineers ignoring or muting alerts (dangerous — real issues get missed)
- Slower response times to genuinely critical issues
- Burnout in on-call engineering teams

Intelligent alerting applies AI/ML techniques (including the anomaly detection and forecasting concepts from Parts 2-3, plus the correlation techniques from Lecture 3) to reduce noise and surface only actionable, high-confidence alerts.

4.2 Key Techniques in Intelligent Alerting

Dynamic Thresholds (from anomaly detection)

Instead of static thresholds, alerts trigger based on learned normal behavior — directly reducing false positives from time-of-day/seasonal patterns (see Part 2).

Alert Suppression & Deduplication

If the same underlying condition triggers 50 similar alerts within a few minutes (e.g., every pod in a deployment reporting the same error), intelligent systems group these into a single alert rather than paging an engineer 50 times. (Covered in depth in Lecture 3.)

Severity Scoring / Prioritization

Rather than treating all alerts equally, ML models score alerts based on: - Business impact of the affected service (from CMDB/topology context — Lecture 1) - Historical pattern (has this alert type led to real incidents before?) - Blast radius (how many downstream systems/users affected)

Predictive Alerting

Using forecasting (Part 3) to alert *before* a threshold is actually crossed: “At current trend, this will breach capacity in 48 hours” rather than waiting until it actually breaches.

Alert Routing & Escalation Intelligence

ML/rules-based systems can learn which team or individual historically resolves a given alert type fastest, and route accordingly, rather than relying on static, manually maintained routing tables.

4.3 Static Alerting vs. Intelligent Alerting — Side-by-Side

Dimension	Static/Traditional Alerting	Intelligent Alerting
Threshold basis	Fixed, manually set	Learned/dynamic, adapts to patterns
Volume	High (many false positives)	Reduced via dedup/correlation
Timing	Reactive (after threshold breach)	Can be predictive (before breach)
Prioritization	Manual/severity tags set once	ML-driven, context-aware scoring
Maintenance	High manual tuning burden	Self-adjusting, but requires monitoring for drift

4.4 Risks and Limitations to Discuss

- **Over-automation risk:** if alert suppression is too aggressive, real issues can be silently dropped — “the boy who cried wolf” in reverse. Always maintain an audit trail of suppressed alerts.
- **Explainability:** engineers need to trust and understand *why* an alert fired (or didn’t) — a “black box” AI alerting system that can’t explain itself will be resisted by operations teams. This is why many tools show the contributing factors/confidence score alongside an ML-driven alert.
- **Cold start problem:** a brand-new service has no historical baseline yet, so dynamic thresholds/intelligent alerting need a fallback (often static thresholds) until enough data accumulates.

4.5 Class Exercise (5 min)

Present three example alerts (e.g., “CPU high,” “5000 identical pod restart alerts in 2 minutes,” “disk projected full in 3 days”). Ask students how an intelligent alerting system would likely handle each differently than a traditional static system.

Part 5: Recap & Discussion (10 min)

Key Takeaways

1. **AIOps** applies AI/ML to IT operations because human-scale review of modern data volumes is no longer feasible.
2. **Anomaly detection** replaces brittle static thresholds with learned, dynamic baselines —

understanding point, contextual, and collective anomalies.

3. **Time-series forecasting** shifts operations from reactive to proactive by predicting future capacity and traffic needs.
4. **Intelligent alerting** combines dynamic thresholds, deduplication, prioritization, and prediction to fight alert fatigue — while requiring careful attention to explainability and the cold-start problem.
5. All of these techniques depend on the foundational context from Lecture 1 (CMDB, topology, observability data) — AI models are only as good as the data and context feeding them.

| Discussion Questions for Wrap-Up

1. Why is anomaly detection typically an unsupervised learning problem rather than a supervised one in most real organizations?
2. What could go wrong if an organization deployed aggressive alert suppression without an audit trail?
3. How would you explain “model drift” to a non-technical manager, and why does it matter for a production AIOps deployment?

| Bridge to Lecture 3

Lecture 2 focused on detecting *individual* anomalies and predicting *individual* metric trends. Lecture 3 addresses the next problem: when a real incident occurs, it rarely produces just *one* alert — it typically produces dozens or hundreds simultaneously across dependent systems. **Event Correlation & Root Cause Analysis** is how AIOps turns that flood of related alerts into a single, clear incident with an identified root cause.

End of Lecture 2 — AI & Machine Learning for IT Operations